Laura Hietala

Oppimistehtävä 1

Tietojenkäsittelyn koulutus
TKMI23KV

29.10.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

GitHub*:* *https://github.com/xamk-so1/oppimisteht-v-1-elahi001xamk*

Perusvaatimukset sovellukselle

Vaatimus: Käyttäjältä kysytään pituus senttimetreissä (huom!) ja paino kilogrammoina kahdella allekkaisella tekstikentällä. Huomaathan, että tekstikentistä poimittu data on aina string-tyyppistä. Muunna stringit number-tyyppiin ennen laskentaa.

Ratkaisu: Toteutus löytyy App.tsx-tiedostosta. Riveillä 6 ja 7 useState-koukku alustaa tilamuuttujat height (pituus) ja weight (paino) tyhjillä merkkijonoilla. Tänne tallennetaan käyttäjän antama data:

```
4  const App : React.FC = () : React.ReactElement => {
5
6    const [height, setHeight] = useState<string>('');
7    const [weight, setWeight] = useState<string>('');
8    const [description, setDescription] = useState<string>('');
9    const [color, setColor] = useState<string>('');
```

Rivillä 41 muunnetaan merkkijonot (stringit) numeroiksi parseFloat-funktiolla. Samalla suoritetaan painoindeksin laskentaa:

```
40
41   const bmi : number = parseFloat(weight) / ((parseFloat(height) / 100) ** 2);
```

Käyttäjältä kysytään pituus senttimetreissä ja paino kilogrammoina seuraavilla kahdella input-tekstikentällä ja tallennetaan tilamuuttujiin riveillä 98-112:

```
98    <input
99      id="input"
100     type="text"
101     placeholder="Pituutesi"
102     onChange={(e) => { setHeight(e.target.value) }}
103     onKeyDown={handleKeyPress}/>
104
105   <br></br>
106
107   <input
108     id="input"
109     type= "text"
110     placeholder="Painosi"
111     onChange={(e) => { setWeight(e.target.value) }}
112     onKeyDown={handleKeyPress}/>
```

Vaatimus: Näiden alapuolella on painonappi "Laske painoindeksi". Napin painaminen suorittaa laskutoimituksen ja tulostaa tuloksen samalle sivulle muodossa "Painoindeksi on: xx.xx". Huomaa pyöristys kahteen desimaaliin.

Ratkaisu: Tämäkin löytyy App.tsx-tiedostosta. Rivillä 116 määritellään Laske painoindeksi-painonappi button-tagien sisällä:

```
115
116   <button id="button" onClick={bmiPrint}>Laske painoindeksi</button>
117
```

Laskutoimitus tapahtuu bmiPrint-funktion sisällä (alkaa riviltä 35 ja jatkuu aina rivin 85 aaltosulkeeseen saakka) ja on sidottu nappiin onClick-tapahtumakäsittelijänä. bmiPrint-funktio suoritetaan, kun nappia painetaan. Nappia painamalla tapahtuu monta bmiPrint-funktion sisällä olevaa asiaa: painoindeksi lasketaan riveillä 35-42 ja se tallennetaan muuttujaan bmiFixed. Lisäksi funktiolla haetaan kuvaileva teksti (rivit 45-77):

```
35  const bmiPrint = () => {
36    if (!isNumeric(height) || !isNumeric(weight)) {
37      setDescription("Syötä kelvolliset arvot sekä pituudelle että painolle.");
38      return;
39    }
40
41    const bmi : number = parseFloat(weight) / ((parseFloat(height) / 100) ** 2);
42    const bmiFixed : string = bmi.toFixed(2);
43
44
45    const getBmiDescription = () => {
46      if (bmi < 15) {
47        return "sairaalloinen alipaino";
48      }
49
50      else if(bmi >= 15 && bmi < 17) {
51        return "merkittävä alipaino";
52      }
53
54      else if(bmi >= 17 && bmi < 18.5) {
55        return "normaalia alhaisempi paino";
56      }
57
58      else if (bmi >= 18.5 && bmi < 25) {
59        return "normaali paino";
60      }
61
62      else if (bmi >= 25 && bmi < 30) {
63        return "lievä ylipaino";
64      }
65
66      else if (bmi >= 30 && bmi < 35) {
67        return "merkittävä ylipaino";
68      }
69
70      else if (bmi >= 35 && bmi < 40) {
71        return "vaikea ylipaino";
72      }
73
74      else if (bmi >= 40) {
75        return "sairaalloinen ylipaino";
76      }
77    }
78
```

Lopuksi päivitetään käyttöliittymä näyttämään laskennan tulos ja kuvaileva teksti. Tämä tapahtuu muuttamalla setDescription-tilamuuttujaa rivillä 84. Tässä setDescription on funktio, joka on peräisin useState-koukusta rivillä 8 (ks. ensimmäinen kuva), ja se päivittää description-tilamuuttujan uudella arvolla, joka on string-tyyppinen merkkijono, joka näyttää lisäksi bmiFixed-muuttujaan tallennetun painoindeksin ja sen kuvailutekstin, joka on tallennettu descriptionResult-muuttujaan:

```
81
82    const descriptionResult = getBmiDescription();
83
84    setDescription(`Painoindeksisi on ${bmiFixed}. Kyseessä on ${descriptionResult}.`);
85  }
86
```

Riveillä 118-120 määritellään vielä napin painamisen tuottaman tuloksen näkyminen käyttöliittymässä div-tagien sisällä:

```
118     <div className={color} id="description">
119         {description}
120         </div>
121
```

toFixed-metodi palauttaa merkkijonon kahden desimaalin tarkkuudella rivillä 42.

```
42      const bmiFixed : string = bmi.toFixed(2);
```

Vaatimus: Riippuen laskutuloksesta, ohjelma tulostaa myös painoindeksiä vastaavan kuvaustekstin (esim. "normaali paino", "lievä ylipaino")

Ratkaisu: Tämän ratkaisu on oikeastaan jo ylemmissä teksteissä ja kuvissa selitetty. Riveillä 45-77 määritellään funktio getBmiDescription, joka ottaa laskettavan painoindeksi parametrina ja palauttaa painoindeksiä vastaavan kuvaustekstin. Funktion sisällä määrittely tapahtuu ehtolausekkeiden avulla. Kuvausteksti palautetaan descriptionResult-muuttujassa rivillä 84 setDescription-funktion kutsussa.

Lisävaatimukset sovellukselle

Lisävaatimus: Laskettu tulos ja painoindeksiä vastaava kuvausteksti sijaitsee div-elementissä, jonka tyyli on määritelty erillisessä css-tiedostossa. Elementin väri/tyyli vaihtuu painoindeksin mukaan. Esimerkiksi vaikea yli- tai alipaino on tyyliltään punainen, merkittävä yli-/alipaino on keltainen ja normaali paino vihreä. Voit valita tyyliratkaisut vapaasti, mutta vähintään tekstin värin ja elementin taustavärin tulee muuttua lasketun painoindeksin mukaisesti.

Ratkaisu: Yllä olevassa kuvassa tiedoston App.tsx riveillä 118-120 div-elementissä sijaitsee laskettu tulos kuvausteksteineen (description). Tässä color liitetään div-elementtiin, ja color päivitetään setColor-funktion kautta lasketun painoindeksin mukaan (rivit 79-80):

```
79      const bmiColor = getBmiColor(bmi);
80      setColor(bmiColor);
81
```

color ja setColor ovat peräisin rivin 9 tilamuuttujien määrittelystä (kuva ensimmäisellä sivulla). Eri painoindeksiluokkien mukaiset tyylit otetaan käyttöön ja muutetaan color-luokan kautta. Sen tyyli on määritelty App.css-tiedostossa:

```
1   .color {
2     color: black;
3     padding: 5px;
4     margin: 5px;
5   }
6
7   .green {
8     max-width: fit-content;
9     color: rgb(2, 63, 2);
10    background-color: rgb(75, 248, 75) ;
11    padding: 5px;
12    margin: 5px;
13  }
14
15  .yellow {
16    max-width: fit-content;
17    color: yellow;
18    background-color: rgb(240, 67, 15);
19    padding: 5px;
20   margin: 5px;
21  }
22
23  .red {
24    max-width: fit-content;
25   color: red;
26   background-color: black;
27   padding: 5px;
28   margin: 5px;
29  }
30
```

getBmiColor-funktio riveillä 11-31 määrittelee ehtolausekkeiden avulla taustavärin (color) sen perusteella, mikä on lasketun BMI:n arvo. Tätä taustaväriä käytetään div-elementin tyyliä ohjaavan CSS-luokan määrittämiseen:

```
11    const getBmiColor = (bmi: number) => {
12      if (bmi < 15) {
13        return 'red';
14      } else if (bmi >= 15 && bmi < 17) {
15        return 'red';
16      } else if (bmi >= 17 && bmi < 18.5) {
17        return 'yellow';
18      } else if (bmi >= 18.5 && bmi < 25) {
19        return 'green';
20      } else if (bmi >= 25 && bmi < 30) {
21        return 'yellow';
22      } else if (bmi >= 30 && bmi < 35) {
23        return 'red';
24      } else if (bmi >= 35 && bmi < 40) {
25        return 'red';
26      } else if (bmi >=40) {
27        return 'red';
28      } else {
29        return 'black';
30      }
31    }
```

Tämä tulos asetetaan color-tilamuuttujaan, joka liitetään CSS-luokkaan, joka ohjaa taustaväriä ja tekstiväriä div-elementissä.